
Adaptive Computation Depth via Learned Token Routing in Transformers

Ahmed Abdelmuniem Abdalla Mohammed
Independent Researcher
ahmed.abdelmuniem@gmail.com
ORCID: 0009-0008-7410-6621

Abstract

Standard transformer architectures apply the same number of layers to every token regardless of contextual difficulty. We present **Token-Selective Attention (TSA)**, a learned per-token gate on residual updates between consecutive transformer blocks. Each gate is a lightweight two-layer multi-layer perceptron (MLP) that produces a continuous halting probability, making the mechanism end-to-end differentiable with 1.7% parameter overhead and no changes to the base architecture. Notably, TSA learns difficulty-proportional routing without any explicit depth pressure: even at $\lambda=0$ (no depth regularisation), the task-loss gradient alone drives the router to skip 20% of token-layer operations. On character-level language modeling, TSA saved 14–23% of token-layer operations (TLOps) across Tiny-Shakespeare and enwik8 at $<0.5\%$ quality loss. At matched efficiency, TSA achieved 0.7% lower validation loss than early exit, and the learned routing transfers directly to inference-time sparse execution for real wall-clock speedup.

Keywords: adaptive computation, token routing, sparse transformers, efficient inference, depth regularisation

1 Introduction

Transformer language models (Vaswani et al., 2017) apply a fixed number of layers to every token in every sequence. This design trades per-token adaptability for architectural simplicity. In practice, the trade-off is costly: a common token in a predictable context requires far less processing than a rare token in a novel construction, yet both receive identical compute at every layer of every forward pass.

The inefficiency is particularly consequential at inference scale. For large deployed models, the dominant cost is the forward pass through all layers for all tokens. If a significant fraction of tokens could exit early without quality loss, the savings would translate directly to reduced latency and throughput gains.

Several approaches have addressed this problem. Graves (2016) introduced Adaptive Computation Time (ACT) for recurrent neural networks (RNNs), accumulating a halting probability across recurrent steps. Dehghani et al. (2019) extended the idea to depth-shared transformer layers with the Universal Transformer. More recently, Raposo et al. (2024) proposed Mixture-of-Depths (MoD), which routes tokens through a fixed subset of layers using hard top- k selection; Bae et al. (2025) introduced Mixture of Recursions, which applies recursive blocks for a learned number of steps per token; and Chen et al. (2025) presented the Inner Thinking Transformer, which inserts additional computation steps at high-stakes positions.

We present **Token-Selective Attention (TSA)**: a continuous soft gate on residual updates, conditioned per token on its current hidden state. The mechanism is architecturally minimal—a two-layer MLP per inter-block gap—and fully differentiable, requiring no straight-through estimators, Gumbel sampling, or reinforcement learning. Our contributions are:

- A simple, differentiable token routing mechanism that gates residual updates softly per token per layer (§2).
- Evidence that routing emerges from the task-loss gradient alone: at $\lambda=0$ (no depth regularisation), the router learns to skip 20% of token-layer operations without any explicit depth pressure (§3.4).
- Cross-dataset validation on character-level language modeling: 14–23% token-layer operations saved across Tiny-Shakespeare and enwik8 at $<0.5\%$ quality loss (§3.2, §3.3).
- Ablations showing robustness to λ across two orders of magnitude (§3.4), quality advantage over early exit at matched efficiency (§3.5), and real wall-clock speedup via sparse inference on commodity hardware (§3.6).

2 Method

2.1 Architecture

Let a pre-norm decoder-only transformer have blocks $f_0, f_1, \dots, f_{L-1}$, where each block applies multi-head self-attention and a feed-forward network (FFN) with residual connections and LayerNorm (Ba et al., 2016). In TSA, a lightweight router r_l is inserted after each block f_l for $l = 0, \dots, L-2$.

Block f_0 is the *stem* and always executes unconditionally: a bare token embedding carries no contextual signal, making a routing decision at step zero uninformative and potentially degenerate. The routing begins after the stem:

$$h \leftarrow f_0(h), \quad p_l = r_l(h), \quad h \leftarrow f_{l+1}(h, p_l), \quad l = 0, \dots, L-2. \quad (1)$$

Figure 1 illustrates the dual-mode mechanism: soft gating during training (differentiable) and hard-threshold sparse execution at inference (real FLOPs savings).

2.2 Router Architecture

Each router is a two-layer MLP with sigmoid output:

$$r_l(h) = \sigma(W_l^{(2)} \text{ReLU}(W_l^{(1)}h + b_l^{(1)}) + b_l^{(2)}), \quad r_l(h) \in (0, 1)^{B \times T}, \quad (2)$$

where the hidden dimension is $d/4$ (floored at 16). Each router adds $d^2/4 + d/2 + 1$ parameters; at $d = 256$, $L = 6$, this totals $\approx 83\text{K}$ on a 4.78M parameter base model (1.7% overhead).

The final bias $b_l^{(2)}$ is initialised to -1.0 , giving $\sigma(-1) \approx 0.27$ at initialisation. This bias prevents early collapse to “halt everything” before the model has learned useful representations.

2.3 Gated Block Update

For each routing decision $l = 0, \dots, L-2$, the gated update of block f_{l+1} is:

$$h \leftarrow h + (1 - p_l) \odot \Delta_{l+1}^{\text{attn}}(h), \quad (3)$$

$$h \leftarrow h + (1 - p_l) \odot \Delta_{l+1}^{\text{ffn}}(h), \quad (4)$$

where $p_l \in (0, 1)^{B \times T}$ is broadcast over the model dimension d , and $\Delta_{l+1}^{\text{attn}}, \Delta_{l+1}^{\text{ffn}}$ are the pre-norm attention and feed-forward residual deltas of block f_{l+1} respectively. When $p_l = 0$, the update is identical to the standard transformer. When $p_l = 1$, the state is unchanged—the block is skipped. The interpolation is smooth, preserving gradient flow through p_l during training.

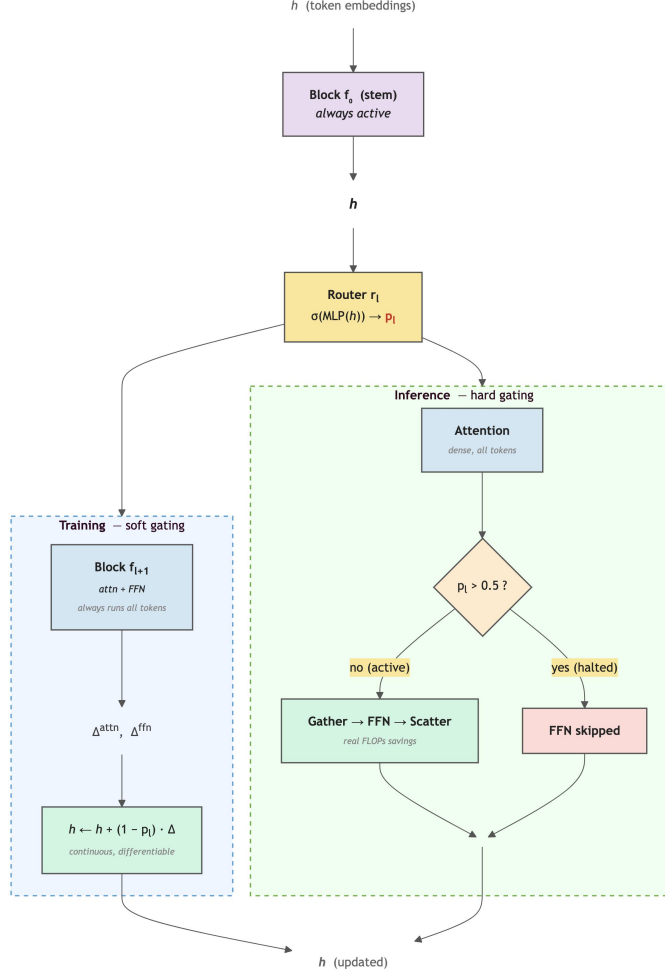


Figure 1: TSA dual-mode architecture. A router r_l reads hidden state h and produces a per-token halting probability p_l . **Left (training):** all tokens always pass through attn + FFN; the residual update is soft-scaled by $(1 - p_l)$, keeping the gate differentiable so the router learns. **Right (inference):** attention remains dense, but tokens with $p_l > 0.5$ skip the FFN entirely via gather/scatter, yielding real FLOPs savings. The stem block f_0 always executes unconditionally.

2.4 Depth Regularisation

Without any incentive to halt, routers default to $p_l \approx 0$ and TSA reduces to a standard transformer with extra parameters. We added a depth regularisation term that gently encourages early halting:

$$\mathcal{L}_{\text{depth}} = \lambda \cdot \frac{1}{L-1} \sum_{l=0}^{L-2} \overline{1 - p_l}, \quad (5)$$

where $\overline{1 - p_l}$ is the mean active fraction at layer l (averaged over batch and sequence position). The total training loss is $\mathcal{L} = \mathcal{L}_{\text{task}} + \mathcal{L}_{\text{depth}}$. We used $\lambda = 0.001$ for language experiments; Section 3.4 demonstrates that TSA is robust across $\lambda \in [0, 0.1]$.

2.5 Compute Metric

We measure compute using **token-layer operations (TLOps)**: for each block, TLOps equals the number of tokens processed at that block. The mean active fraction across routing decisions is:

$$\alpha = \frac{1}{L-1} \sum_{l=0}^{L-2} \frac{1}{1-p_l}. \quad (6)$$

TLOps savings relative to the fixed-depth baseline are:

$$\Delta = 1 - \frac{1 + (L-1)\alpha}{L}. \quad (7)$$

The stem block (always active) is included in both numerator and denominator, making Δ a conservative estimate.

Note on training compute. During training, all layers execute fully: the gate scales residual updates but does not skip computation. TLOps therefore measures the effective contribution of each layer to the final representation, not actual FLOPs saved. At inference, sparse-TSA (Section 3.6) exploits low-contribution positions via gather/scatter to achieve real compute savings.

3 Experiments

3.1 Synthetic Algorithmic Tasks

Setup. We used decoder-only transformers trained on **copy** and **sort** tasks over length-10 sequences from a 32-token vocabulary. Inputs followed the format [BOS] src [SEP] tgt [EOS] with loss masked on source tokens. Both baseline and TSA used $d=128$, $L=6$, $H=4$, $d_{\text{ff}}=512$ (baseline: 1.20M params; TSA: 1.22M params, +1.7%). Training employed AdamW (Loshchilov and Hutter, 2019) with $\beta=(0.9, 0.95)$, $\text{lr}=3 \times 10^{-4}$, $\lambda_{\text{wd}}=0.1$ for 10K gradient steps on 10K training sequences. We report token-level sequence accuracy on 1K held-out sequences.

Table 1: Synthetic Task Results ($d=128$, $L=6$, Toy Vocabulary)

Task	Model	Accuracy	Perplexity	Active Frac α	TLOps saved [†]
Copy	Baseline	1.0000	1.012	1.000	—
Copy	TSA	1.0000	1.012	0.341	54.9%
Sort	Baseline	0.9915	1.079	1.000	—
Sort	TSA	0.9878	1.087	0.730	22.5%

[†] TLOps saved = $1 - (1 + (L-1)\alpha)/L$, including the mandatory stem block.

Results. The routing pattern directly reflected task difficulty. Copy is an identity mapping: the router learned that nearly all tokens were fully determined after the stem block ($\alpha = 0.341$; 54.9% overall TLOps saved). Sort requires comparison and permutation, yielding $\alpha = 0.730$ —more compute where the task genuinely demanded it. This difficulty-proportional allocation emerged without any explicit supervision about task identity or difficulty.

3.2 Character-Level Language Modeling

Setup. We trained on Tiny-Shakespeare (Karpathy, 2015) (1.1M characters, 65-char vocabulary, 80/10/10 train/val/test split). Both models used $d=256$, $L=6$, $H=8$, $d_{\text{ff}}=1024$, context length 128. Training employed AdamW with cosine learning rate schedule, batch size 64, for 5,000 gradient steps (baseline: 4.78M params; TSA: 4.86M params, +1.7%). Token embeddings were initialised without a padding index: character index 0 is the newline character (~8% of the corpus), whose embedding gradient must not be zeroed.

Results. TSA achieved $\alpha = 0.726$: 22.8% of token-layer operations saved at a cost of 0.006 nats (+0.4%) in validation loss. Both models reached all convergence thresholds at identical step counts, indicating TSA did not impede convergence (Figure 2). The TSA curve lies consistently to the left on the compute axis, confirming that savings remained stable throughout training.

Table 2: Language Modeling Results ($d=256$, $L=6$, Tiny-Shakespeare)

Model	Params	Val Loss	BPC	α	TLOps saved
Baseline	4,782,336	1.4422	2.0807	1.000	—
TSA	4,864,901	1.4482	2.0893	0.726	22.8%

Val loss increase: +0.006 nats (+0.4% relative). BPC = bits-per-character.

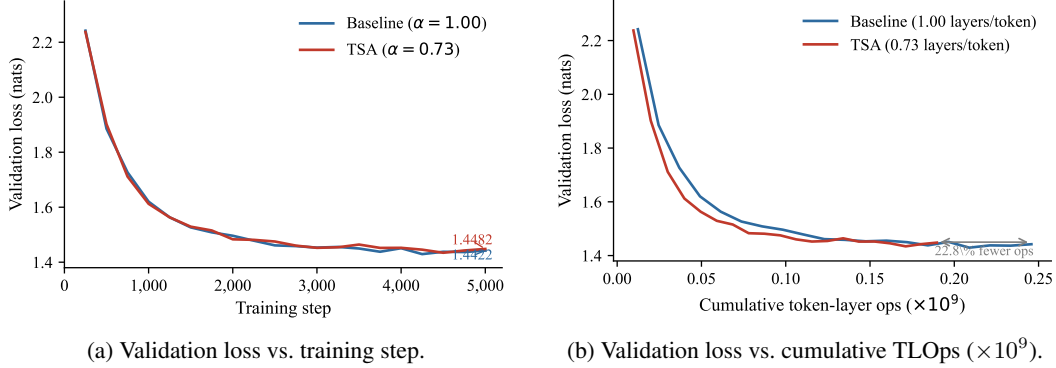


Figure 2: TSA (red) and Baseline (blue) on Tiny-Shakespeare. Left: equivalent convergence speed. Right: TSA reaches the same loss for 22.8% fewer token-layer operations.

3.3 Cross-Dataset Validation: enwik8

To test whether TSA generalises beyond a single corpus, we trained on enwik8 (Hutter, 2006): the first 10^8 bytes of English Wikipedia (raw XML, 6,064 unique characters). This corpus is substantially more diverse than Shakespeare—it contains markup, multilingual text, tables, and mathematical notation. We used $d=256$, $L=6$, $H=8$, $d_{\text{ff}}=1024$, context length 256, batch size 64, for 5,000 steps (6.35M params baseline; 6.43M TSA). Experiments were conducted on Apple M1 Pro using MLX (Apple Machine Learning Research, 2023).

Table 3: enwik8 Results ($d=256$, $L=6$, Context 256)

Model	Val Loss	BPC	α	TLOps saved	ms/step
Baseline	1.2826	1.8504	1.000	—	585
TSA	1.2774	1.8429	0.833	13.9%	599

TSA quality vs. Baseline: -0.4% (TSA is marginally better; within noise).

TSA achieved $\alpha = 0.833$ on enwik8, more conservative than Shakespeare’s $\alpha = 0.726$. The router allocated more compute on the structurally diverse Wikipedia corpus while still saving 13.9% of TLOps at no quality cost. Both conditions reached all convergence thresholds (≤ 2.5 , ≤ 2.0 , ≤ 1.8 BPC) at identical steps (500, 750, 1,000). The cross-dataset result confirms that the routing mechanism learned a content-dependent signal rather than overfitting to corpus-specific patterns. Training curves are presented in Figure 5 (Appendix).

3.4 Ablation: Depth Regularisation Sensitivity

We swept $\lambda \in \{0, 0.001, 0.005, 0.01, 0.05, 0.1, 0.5\}$ on Tiny-Shakespeare with all other hyperparameters fixed at the values in §3.2. Figure 3a shows the quality-efficiency Pareto curve; full results are presented in Table 6 (Appendix).

Three findings emerged. (i) $\lambda=0$ *still routes*: without any explicit depth pressure, the router learned to save 20.4% of TLOps ($\alpha = 0.755$) via the task-loss gradient alone. This is the central finding: the gating multiplication $h \leftarrow (1-p_l) \odot \Delta$ provides an intrinsic learning signal—when a layer’s residual update is noisy or redundant, the gradient favours increasing p_l to attenuate the update, even without regularisation. The router thus acts as a learned noise gate. (ii) *Robustness*: across

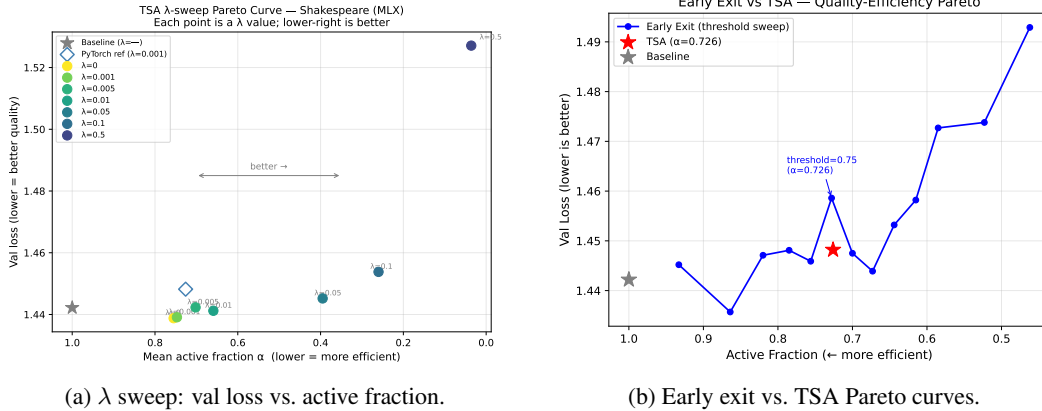


Figure 3: Ablation studies on Tiny-Shakespeare. **Left:** TSA is robust across $\lambda \in [0, 0.1]$; quality range is 0.015 nats (1.04%). Even $\lambda=0$ produces meaningful routing. **Right:** TSA (red star) dominates the early exit threshold sweep (blue) at matched $\alpha \approx 0.726$.

$\lambda \in [0, 0.1]$, the quality range was only 0.015 nats (1.04% relative)—TSA does not require precise tuning of λ . (iii) $\lambda=0.05$ is *Pareto-optimal*: 50.4% TLOps saved at $<0.5\%$ quality loss, 2.4 times as efficient as the default $\lambda=0.001$ with negligible quality cost. The stability boundary is $\lambda < 0.5$; at $\lambda=0.5$ the active fraction collapsed to 0.036 and quality degraded by 5.9%.

3.5 Ablation: Comparison With Early Exit

The early exit approach (Elbayad et al., 2020) is the canonical inference-time baseline for adaptive-depth transformers. We trained an early exit model with N auxiliary exit classifiers (one per block, tied embedding output heads, uniform mean cross-entropy loss across all exits) on the same Shakespeare setup. At inference, a token exited when its maximum softmax probability exceeded a confidence threshold. We swept 13 thresholds and selected the one yielding $\alpha \approx 0.726$ to match the TSA operating point. Full threshold data are presented in Table 7 (Appendix).

Table 4: Comparison at Matched Active Fraction ($\alpha \approx 0.726$, Shakespeare)

Model	Val Loss	BPC	α	TLOps saved
Baseline	1.4422	2.0807	1.000	—
TSA	1.4482	2.0893	0.726	22.8%
Early Exit	1.4586	2.1043	0.728	22.7%

TSA’s router operates at both train and inference time, learning routing decisions end-to-end via the task-loss gradient. Early exit trains identically to the baseline and applies routing only at inference via a separate confidence threshold. At matched active fraction, TSA achieved 0.71% lower validation loss than early exit (Table 4), suggesting that end-to-end learned routing produces better routing decisions than post-hoc confidence thresholding. At more conservative thresholds (higher α , fewer tokens skipped), early exit quality improves—as expected, since fewer routing decisions are made—but the comparison at matched α isolates routing quality from routing aggressiveness (Table 7).

3.6 Wall-Clock Throughput

Soft gating (§2) multiplies all residuals by $(1-p_l)$ but does not skip any computation. To translate TLOps savings to wall-clock speedup, we implemented **sparse-TSA**: at inference, tokens with $p_l > 0.5$ were excluded from the FFN via gather/scatter operations; attention computation remained dense to preserve exact key-value (KV) semantics, though the attention residual update was gated by the same binary mask as the FFN. We benchmarked on Apple M1 Pro using MLX (Apple Machine Learning Research, 2023) with batch=64, seq=256, 30 warmup + 200 timed forward passes per configuration. Full data are presented in Table 8 (Appendix).

Table 5: Wall-Clock Throughput (M1 Pro, MLX, Batch=64, Seq=256)

Condition	α	TLOps saved	ms/step	Speedup
Baseline	—	—	113.2	1.000×
Soft-TSA	any	—	~114.4	~0.99×
Sparse-TSA	0.726	22.8%	110.7	1.023×
Sparse-TSA	0.833	13.9%	113.1	1.000×

Soft-TSA overhead ($\sim 1\%$) is flat across all α —the router is negligible cost. Sparse-TSA speedup requires batch ≥ 64 .

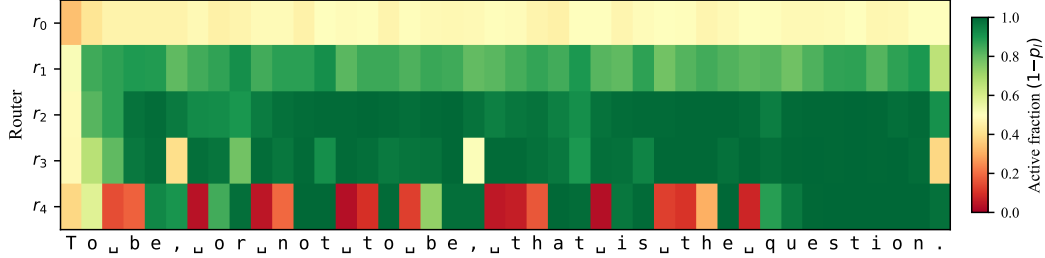


Figure 4: Per-token active fraction ($1 - p_l$) across five routing decisions on a Shakespeare passage. Green: fully active; red: nearly halted. Early routers stay permissive; later routers selectively gate predictable tokens (spaces, punctuation, common characters) while preserving computation for content-bearing positions.

Soft gating added $\sim 1\%$ overhead regardless of α —the router represented negligible cost. Sparse-TSA was faster than the baseline for $\alpha \leq 0.83$: 2.3% speedup at $\alpha=0.726$ (Shakespeare), break-even at $\alpha=0.833$ (enwik8). The speedup required batch ≥ 64 ; at batch=1, CPU-GPU synchronisation dominated.

4 Analysis and Limitations

What did the router learn? On synthetic tasks, routing was interpretable: copy halted maximally (identity requires no deep computation); sort halted moderately (comparison needs depth). On language, the router was more conservative on enwik8 ($\alpha = 0.833$) than Shakespeare ($\alpha = 0.726$), consistent with Wikipedia’s greater structural diversity.

Figure 4 shows per-token routing decisions across a sample Shakespeare passage. Early routers (r_0, r_1) keep most tokens active, while later routers (r_3, r_4) exhibit selective gating: punctuation, spaces, and predictable characters are attenuated more aggressively than content-bearing characters in mid-word positions. This confirms that the router learns a difficulty-sensitive signal rather than uniform layer skipping.

Limitations.

- *Scale.* All experiments used $\approx 5\text{--}6\text{M}$ parameters; scaling behaviour at $10\text{M--}100\text{M}$ is unknown.
- *Batch-size dependence.* Sparse-TSA wall-clock speedup required batch ≥ 64 ; custom Metal kernels would likely eliminate this.
- *No attention sparsity.* Only FFN was sparsified; block-sparse attention could yield larger savings but would require retraining.
- *Preliminary routing analysis.* Figure 4 shows qualitative patterns; quantitative analysis by token type (e.g., frequency, entropy) remains future work.

5 Related Work

Adaptive Computation. Graves (2016) introduced ACT for RNNs; Dehghani et al. (2019) extended this approach to weight-shared layers with the Universal Transformer. TSA differs in using separate blocks and per-layer soft gates rather than an accumulated budget. Raposo et al. (2024) proposed Mixture-of-Depths, which uses hard top- k routing; Bae et al. (2025) introduced Mixture of Recursions, which applies recursive blocks for a learned number of steps. Both employ discrete routing; TSA uses a continuous gate.

Early Exit. Elbayad et al. (2020) proposed the Depth-Adaptive Transformer, which trains auxiliary classifiers and exits tokens at inference based on output confidence. Training cost equals the baseline. Our ablation (§3.5) showed that TSA achieved better quality at matched efficiency, suggesting that end-to-end learned routing outperforms post-hoc confidence thresholding.

Other Approaches. Chen et al. (2025) proposed the Inner Thinking Transformer, which augments compute at hard positions (complementary to TSA). Fedus et al. (2022) introduced Switch Transformers, which route tokens to different expert FFNs, varying width rather than depth.

6 Conclusion

TSA reduced token-layer operations by 14–23% on character-level language modeling and up to 55% on synthetic tasks, at $<0.5\%$ quality cost across two language corpora. The router learns difficulty-proportional allocation from the task-loss gradient alone, producing meaningful routing even at $\lambda=0$. The mechanism adds 1.7% parameters, proved robust to λ across two orders of magnitude, achieved better quality than early exit at matched efficiency, and translated to real wall-clock speedup via sparse inference at batch ≥ 64 . Scaling to 10M+ parameters and per-position routing analysis are ongoing.

References

- Apple Machine Learning Research (2023). MLX: An array framework for Apple silicon. <https://github.com/ml-explore/mlx>.
- Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. In *arXiv preprint arXiv:1607.06450*.
- Bae, S., Hwang, Y., Noh, J., Kim, M., Yoo, K. H., and Yoo, C. D. (2025). Mixture of recursions: Learning dynamic recursive depths for adaptive token-level computation. In *International Conference on Machine Learning (ICML)*.
- Brown, T. B. et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems (NeurIPS)*, 33.
- Chen, Y., Wang, J., Luo, Z., Wang, X., Li, G., Zhao, Z., Zeng, X., Liu, T., Ding, B., and Zhou, J. (2025). Inner thinking transformer: Leveraging dynamic depth scaling to foster adaptive internal thinking. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., and Kaiser, L. (2019). Universal transformers. In *International Conference on Learning Representations (ICLR)*.
- Elbayad, M., Gu, J., Grave, E., and Auli, M. (2020). Depth-adaptive transformer. In *International Conference on Learning Representations (ICLR)*.
- Fedus, W., Zoph, B., and Shazeer, N. (2022). Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39.
- Graves, A. (2016). Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*.
- Hutter, M. (2006). The hutter prize. <http://prize.hutter1.net/>.

Karpathy, A. (2015). char-rnn. <https://github.com/karpathy/char-rnn>.

Loshchilov, I. and Hutter, F. (2019). Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*.

Press, O. and Wolf, L. (2017). Using the output embedding to improve language models. *Proceedings of the Conference of the European Chapter of the Association for Computational Linguistics (EACL)*.

Raposo, D., Ritter, S., Richards, B., Lillicrap, T., Humphreys, P. C., and Santoro, A. (2024). Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30.

A Implementation Details

Weight Initialisation. Token and positional embeddings: $\mathcal{N}(0, 0.02^2)$. Residual projections (attention output and final feed-forward layer): $\mathcal{N}(0, (0.02/\sqrt{2L})^2)$, following GPT-3 (Brown et al., 2020). Router final bias: -1.0 . Weight tying between token embedding and output head followed Press and Wolf (2017).

Optimiser. AdamW (Loshchilov and Hutter, 2019) with $\beta = (0.9, 0.95)$, $\lambda_{wd} = 0.1$ on all parameters except biases, LayerNorm parameters, and embeddings (which used $\lambda_{wd} = 0$). The MLX implementation applies weight decay uniformly to all parameters, as MLX does not support per-parameter decay groups. This affects the enwik8 experiments (Table 3) and the λ sweep (Table 6).

Causal Masking. Standard upper-triangular causal attention mask was used. Routing decisions were computed from hidden states after the causal attention sublayer and did not depend on the mask structure.

Character-Level Tokenisation. Characters were mapped to integer indices via a sorted vocabulary of unique characters in the training corpus (65 characters in Tiny-Shakespeare; 6,064 unique characters in enwik8). Token index 0 mapped to the newline character in Shakespeare; the embedding for this token was initialised normally (no `padding_idx` was set) to avoid zeroing gradients for $\approx 8\%$ of corpus tokens. enwik8 used the raw byte distribution with no preprocessing beyond vocabulary construction.

B enwik8 Training Curves

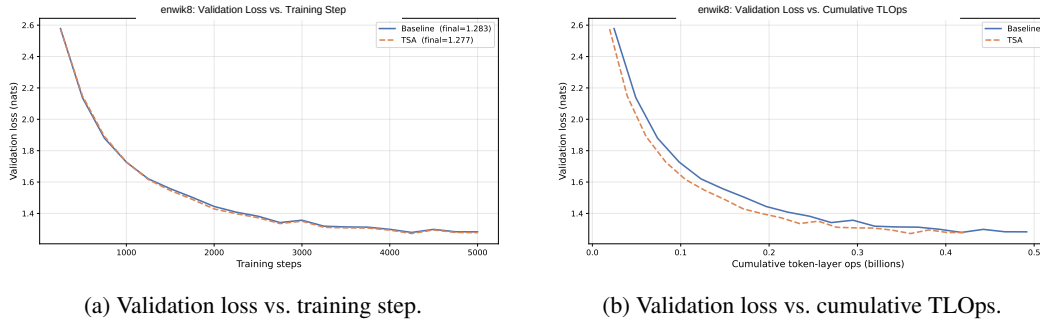


Figure 5: TSA (red) and Baseline (blue) on enwik8. Convergence speed is identical; TSA reaches the same loss for 13.9% fewer token-layer operations.

C Full λ Sweep Results

Table 6: Full λ Sweep on Tiny-Shakespeare (5,000 Steps, Batch=64, Ctx=128)

λ	Val Loss	BPC	α	TLOps saved	Note
— (Baseline)	1.4422	2.0807	1.000	0%	baseline
0	1.4388	2.0757	0.755	20.4%	routes w/o pressure default
0.001	1.4391	2.0762	0.747	21.1%	
0.005	1.4423	2.0808	0.702	24.8%	
0.01	1.4412	2.0793	0.659	28.4%	
0.05	1.4452	2.0849	0.395	50.4%	Pareto-optimal
0.1	1.4538	2.0974	0.260	61.7%	degraded (+5.9%)
0.5	1.5271	2.2031	0.036	80.3%	

All λ sweep experiments used MLX on Apple M1 Pro; within-sweep comparisons are framework-consistent.

The $\lambda=0.001$ result differs from Table 2 (PyTorch, MPS) by 0.63% in validation loss due to framework and RNG differences; cross-framework comparisons should use the sweep-internal baseline row above, not Table 2.

D Full Early Exit Threshold Sweep

Table 7: Early Exit Threshold Sweep on Tiny-Shakespeare (Post 5,000 Steps Training)

Threshold	Val Loss	BPC	α	TLOps saved
0.99	1.4452	2.0849	0.933	5.6%
0.95	1.4357	2.0713	0.864	11.3%
0.90	1.4471	2.0877	0.820	15.0%
0.85	1.4481	2.0891	0.785	17.9%
0.80	1.4459	2.0861	0.756	20.3%
0.75	1.4586	2.1043	0.728	22.7%
0.70	1.4475	2.0883	0.700	25.0%
0.65	1.4439	2.0831	0.673	27.3%
0.60	1.4532	2.0965	0.644	29.7%
0.55	1.4582	2.1037	0.615	32.0%
0.50	1.4727	2.1247	0.585	34.6%
0.40	1.4738	2.1263	0.523	39.8%
0.30	1.4929	2.1538	0.462	44.9%

Bold row matches the TSA operating point ($\alpha \approx 0.726$). Full model (no exit): val loss = 1.4450, identical to Baseline training cost.

E Full Wall-Clock Data

Table 8: Active-Fraction Sweep (M1 Pro, MLX, Batch=64, Seq=256)

α	TLOps saved	Baseline (ms)	Sparse-TSA (ms)	Speedup
0.100	75.0%	113.2	90.8	1.246×
0.200	66.7%	113.2	93.8	1.207×
0.300	58.3%	113.2	96.9	1.168×
0.400	50.0%	113.2	100.3	1.129×
0.500	41.7%	113.2	103.2	1.096×
0.600	33.3%	113.2	105.9	1.068×
0.700	25.0%	113.2	110.3	1.026×
0.726	22.8%	113.2	110.7	1.023×
0.800	16.7%	113.2	112.4	1.006×
0.833	13.9%	113.2	113.1	1.000×
0.900	8.3%	113.2	115.3	0.981×
1.000	0.0%	113.2	118.3	0.956×

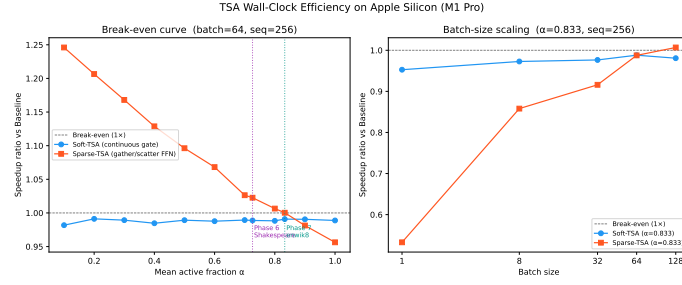


Figure 6: Wall-clock speedup vs. active fraction. Sparse-TSA is faster than Baseline for $\alpha \leq 0.83$. Vertical dashed lines mark the Shakespeare ($\alpha = 0.726$) and enwik8 ($\alpha = 0.833$) operating points.

Table 9: Batch-Size Scaling at $\alpha = 0.833$ (M1 Pro, MLX, Seq=256)

Batch	Baseline (ms)	Sparse-TSA (ms)	Speedup
1	3.3	6.2	0.533×
8	16.1	18.8	0.858×
32	56.7	61.9	0.916×
64	113.2	113.1	1.000×
128	225.9	224.3	1.007×

At $\alpha=0.833$, sparse-TSA breaks even at batch=64 and achieves speedup at batch=128; at batch=1, CPU-GPU syncs dominate.